

作品概要 / 何を作ったか

DirectX 12 と C++20 で制作した Windows 向けゲームエンジンと、3Dサバイバルシューティング。概要羅列ではなく、実装証拠を中心に説明する。

KuboEngine / DirectXGame

`engine/` に描画・入力・音声・リソース・scene管理を置き、`game/directxgame/` にタイトル、ゲームプレイ、リザルト、敵、武器、HUD、演出、debug、CSV調整を分離しました。

Gameplay screenshot

提出前に実ゲーム画面へ差し替え

深掘り 1

敵管理 / OBB衝突 / 空間分割 / Telemetry

深掘り 2

CSVデータ駆動 / 選定理由 / strict parse

深掘り 3

DirectX 12 FrameContext / SRV管理

検証

CPU regression / scene stress / PIX preflight

判断 / 注意

FPSやGPU時間は未計測のため、資料では性能改善値として主張しない。現時点で出せる数値は、scene stress と soft-cap telemetry に限定する。

担当範囲 / 技術スタック

個人制作として、ゲームロジックだけでなくDX12描画基盤、scene遷移、resource管理、debug UI、CSV調整、CPU test、stress runnerまで担当。

Engine

```
`Framework`  
`DirectXCommon`  
`SrvManager`  
input / audio / camera  
particle / model / texture
```

Game

```
`TitleScene`  
`PlayScene`  
`ResultScene`  
player / enemy / weapon  
HUD / effects / debug
```

Data / Tools

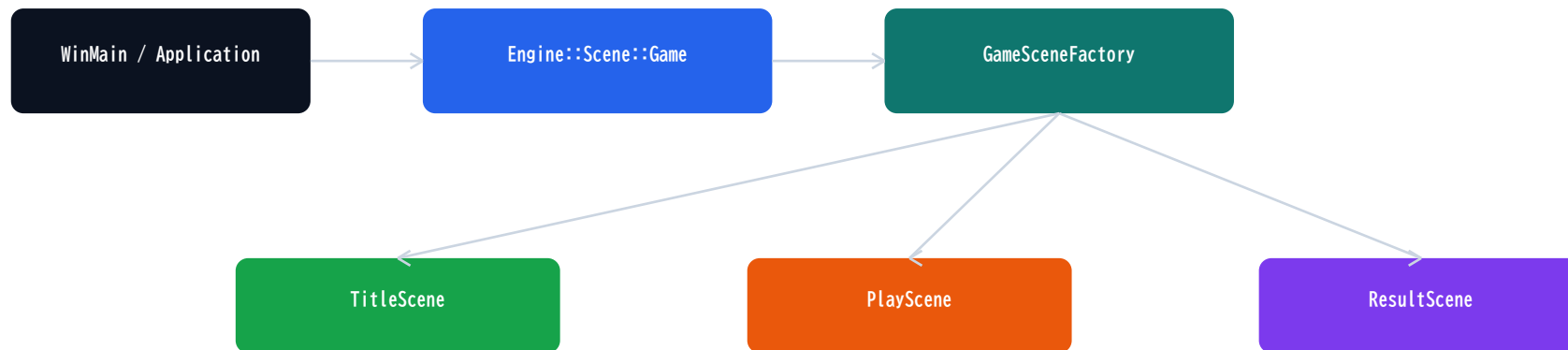
```
`Resources/DirectXGame/data/*.csv`  
CPU regression checks  
scene transition stress  
PIX preflight
```

使用技術

- C++20 / MSVC / Visual Studio solution / Windows 10-11 x64
- DirectX 12 / HLSL / DirectXTex / Assimp
- Dear ImGui / ImGuizmo / ImPlot / imgui-node-editor / FontAwesome icon font
- 設計対象: rendering, frame resources, descriptors, gameplay flow, CSV tuning, debug telemetry

全体アーキテクチャ

`main.cpp` は起動と例外処理に絞り、実行本体は `Engine::Scene::Game` へ委譲。scene生成は `GameSceneFactory` が担当する。



コード証拠: main.cpp

```
auto directXGame = std::make_unique<Engine::Scene::Game>(
    std::make_unique<DirectXGame::GameSceneFactory>(),
    DirectXGame::SceneId::kTitle);
std::unique_ptr<Engine::Base::Framework> game = std::move(directXGame);
game->Run();
```

設計意図

起動処理、共通Framework、ゲーム固有sceneの変更理由を分ける。新しいsceneを追加する場合も、main loop を触らず factory 側へ閉じられる。

現行ゲーム構成

`PlayScene` はgameplayのcomposition root。内部処理は flow, player, enemy, HUD, effect, debug へ分割している。

GameplayFlowController	Start / Playing / BossIntro / Boss / BossDefeated / Paused / LevelUp / Dead
PlayerManager	HP, EXP, Level, weapon upgrade の窓口
EnemyManager	enemy, EXP Orb, collision, boss state
GameplayHudPresentation	HP / EXP / timer / minimap などのHUD
GameParticleEffects	level-up, combat, trail などの粒子演出
DebugContext / DebugUI::*	freeze, debug state, editor連携

判断 / 注意

旧資料に残っていた古いscene/session/debug系の名称は、現行コードの `PlayScene`, `GameSession`, `GameSceneFactory`, `DebugContext / DebugUI::\*` へ更新済み。

深掘り1: 敵管理と OBB 衝突

敵、通常弾、衛星弾、ドローン弾、EXP Orb が同時に増える場面で、候補数と一時確保が問題になる。

OBB / Collider debug screenshot

提出前に実ゲーム画面へ差し替え

- `EnemyCollisionContext` が `spatialMap`, `activeEnemies`, `nearbyEnemies` を保持。
- `EnemyCollisionSystem` が `RebuildContext`, `CheckCollisions`, `ApplyAreaDamage`, `ResolveEnemySeparation` を担当。
- OBBはnarrow phaseとして使い、見た目の回転やスケールと当たり判定のずれを減らす。

コード証拠: GameplayRules::MakeCellKey

```
inline uint64_t MakeCellKey(int32_t cellX, int32_t cellZ)
{
    return (static_cast<uint64_t>(
        static_cast<uint32_t>(cellX)) << 32) |
        static_cast<uint32_t>(cellZ);
}
```

判断 / 注意

負座標のcellも扱うため、`uint32\_t` 経由で `uint64\_t` にpackする。負値を左shiftする形は未定義動作の危険があるため、この形にした。

深掘り1: Telemetry と負荷監視

`SoftCapTelemetry` は enemy数、kill数、EXP Orb数、弾数、prune数、particle数をsnapshotとして取得する。

level 8	killCount 209	expOrbCap 160
expOrbPeak 161	expOrbPrunes 31	expOrbPrunesPerMinute 10.2245
normalBulletCap 96	droneBulletCap 64	particleCount 75

判断 / 注意

この数値はBefore/After性能改善値ではなく、上限管理と実行中監視の証拠。空間分割の効果を示すには、総当たり版との比較benchmarkを追加する。

ImGui telemetry / debug screenshot

提出前に実ゲーム画面へ差し替え

深掘り2: CSV データ駆動の技術選定

武器レベル、敵パラメータ、UI座標のような表形式データを、コードからCSVへ分離した。

CSV

良い点: 表形式、Excel編集、Git差分が見やすい
判断: 採用

JSON

良い点: 階層構造に強い
判断: 今回の中心データでは冗長

独自バイナリ

良い点: 読み込み効率に強い
判断: 手編集とdebugが難しい

C++直書き

良い点: 型安全にしやすい
判断: 調整ごとにビルドが必要

現行CSV

- `playerStatus.csv`, `weaponUpgradeSettings.csv`, `enemyTypes.csv`, `enemySpawnSettings.csv`
- `levelupWeights.csv`, `ui\_layout\_title/hud/levelup/pause/result.csv`
- `resource\_manifest.csv`, `debug\_tuning.csv`, `soft\_cap\_telemetry.csv`

深掘り2: CSV の弱点と対策

CSVは編集しやすい反面、型情報が弱い。`1.0x` や `nan` の混入を防ぐため、strict parse とCPU testで境界を固定した。

CsvReader.cpp

```
const float result = std::stof(text, &parsedLength);
if (parsedLength != text.size() || !std::isfinite(result)) {
    throw std::invalid_argument("not a finite float");
}
```

cpu_regression_checks.cpp

```
RequireThrows([] {
    CsvReader::ParseFloat("1.0x", "test");
}, "float trailing garbage");
RequireThrows([] {
    CsvReader::ParseFloat("nan", "test");
}, "float nan rejection");
```

CSV file screenshot

提出前に実ゲーム画面へ差し替え

設計判断

CSVの弱点を隠さず、parse側で壊れ方を決める。採用理由は「編集しやすさ」であり、型安全性はCPU regression checksで補う。

深掘り3: DirectX 12 リソース寿命管理

CPU側で不要に見えるupload resourceでも、GPUがまだ参照している可能性がある。FrameContextで寿命境界を明示した。

```
struct FrameContext
{
    ComPtr<ID3D12CommandAllocator> commandAllocator;
    ComPtr<ID3D12Resource> uploadArena;
    std::vector<ComPtr<ID3D12Resource>> deferredReleaseResources;
    std::byte* uploadCpuAddress = nullptr;
    size_t uploadOffset = 0;
    uint64_t fenceValue = 0;
};
```



設計意図

command allocator, upload arena, deferred release resources, fence value をframeごとに持つ。frame再利用時にGPU完了を確認し、deferredReleaseResourcesをclearする。

判断 / 注意

PIX timingは未計測のため、ここでは定量性能値として扱わない。現時点では「壊れにくいリソース寿命設計」として説明する。

改善履歴 / 問題解決能力

提出前レビューで見つけた危険箇所を、動作確認だけで済ませず、設計として直した。

問題	原因	修正	結果
SRVが増え続ける危険	descriptor ownerが曖昧	`SrvManager`へ集約、free/high-watermark追加	scene stressで105/105
upload resource寿命が曖昧	GPU完了前に解放され得る	`FrameContext` + deferred release	frame単位で寿命管理
CSV不正値の危険	CSVは型情報が弱い	strict parse + CPU tests	`nan`, overflow等を検出
乱数の再現性不足	process global random	run seedを`GameSession`へ	debug/testで再現しやすい
debug名の冗長化	`Gameplay*Debug*`が増えた	`DebugUI::*` namespaceへ整理	stateless処理を軽量化

資料に載せる数値は、現行プロジェクト内に残っているログとテストに限定する。

scene_transition_stress.txt

```
status=PASS
targetCycles=3
runCount=3
titleVisits=3
gameVisits=3
resultVisits=3
maxSrvUsed=105
maxSrvHighWatermark=105
```

Scene stress の意味

``TitleScene -> PlayScene -> ResultScene`` を自動遷移し、scene訪問回数とSRV使用量を記録する。descriptor使用量がscene遷移で増え続けないかを見る検証。

CPU regression checks

cell key packing
gauge clamp
strict CSV parse
weapon interval normalization
run seed derivation
strong SoundHandle

PIX preflight

status=READY
PIX installed=True
CR-012 dynamic texture load latency
CR-026 shadow pass timing

判断 / 注意

PIXは準備完了まで。GPU pass timeやdraw-call breakdownは未計測なので、提出資料では今後の改善に置く。

今後の改善 / 提出前に追加する証拠

現時点の資料は、古い内容を現行コードに合わせて更新した版。最後に、企業提出前に追加すべき証拠を明記する。

計測を追加

collision benchmark
総当たり vs 空間分割
candidate数 / narrow phase数
collision ms をgame内記録

画像を追加

gameplay画面
OBB debug画面
ImGui telemetry
CSV変更反映画面

DX12証拠を追加

PIX capture
shadow pass timing
dynamic texture load latency
SRV usage graph

締めの文章

KuboEngineでは、描画APIを使うだけでなく、リソース寿命、scene遷移、調整データ、負荷監視、検証を自分で扱いました。次の段階では、実装した設計を計測値と画面証拠でさらに説明できる資料にします。